Lab 2D: Package and Ship - Plugins & Marketplaces

Duration: 35 min

After: Plugins & Marketplaces lecture (Day 2, Block 4)

Prerequisites: Labs 2A-2C complete (skill + hooks + command in `.claude/`)

Objective

Scaffold a plugin with `claude plugin init`, package your toolkit (skill + hook + command) as a distributable plugin, publish it to a local/team marketplace, and install it back - plus a 2-minute teaser of `claude mcp login` for tomorrow.

Deliverable (toolkit chain 8 of 12)

Toolkit artifact #5: a **plugin manifest** (`.claude-plugin/plugin.json`) wrapping your skill, hook, and command - published to a local marketplace and installable by a teammate. Your toolkit repo is now complete: CLAUDE.md + skill + hook + command + plugin manifest.

> ***START MARKER:*** `/clear` *done;* `/skills` *shows code-review;* `/review` *works.*

Fell behind? Run:

```
Set up this project with: (1) a code-review skill at .claude/skills/code-review/ (current
frontmatter incl. allowed-tools), (2) a PostToolUse prettier hook and PreToolUse secret
blocker in .claude/hooks/ registered in .claude/settings.json, (3) a /review command in
.claude/commands/, (4) CLAUDE.md. Make scripts executable.
```

Exercise 1: Explore the Official Marketplace (5 min)

```
/plugin
```

Browse `claude-plugins-official` - the auto-available official marketplace (~101 plugins as of March 2026; the wider ecosystem counts 23,000+ skills and 2,600+ marketplaces).

> ***Trust model:*** *plugins are highly-trusted components - a plugin can bundle skills, hooks, commands, AND MCP servers that all run with your permissions. Treat plugin installs like adding a developer dependency: read what's inside before installing. That's why you inspect before you install.*

Pick any plugin (e.g., skill-creator) and inspect it before installing:

```
Show me what the skill-creator plugin from claude-plugins-official contains - its skills,
commands, hooks, and MCP servers - before I decide to install it.
```

Exercise 2: Scaffold Your Plugin - claude plugin init (10 min)

Step 1 - exit Claude (`exit`) and scaffold from the shell:

```
cd ..
mkdir dashboard-toolkit-plugin && cd dashboard-toolkit-plugin
claude plugin init
```

Expected output marker: an interactive scaffold creating the plugin skeleton, including `.claude-plugin/plugin.json` (name, description, version, author) and empty `skills/`, `commands/`, `hooks/` directories. Accept name `dashboard-toolkit`, version `0.1.0`.

Step 2 - inspect what was generated:

```
find . -type f | head -20
cat .claude-plugin/plugin.json
```

Expected output marker: valid JSON manifest with your plugin name.

Exercise 3: Package Your Toolkit (8 min)

Start Claude in the plugin folder (`claude`) and move your artifacts in:

```
Copy my toolkit from ../business-dashboard into this plugin:
1. .claude/skills/code-review/ (SKILL.md + references/) into skills/code-review/
2. .claude/hooks/post-edit-format.sh and pre-write-check.sh into hooks/, plus a
hooks/hooks.json declaring the PostToolUse (matcher "Write|Edit") and PreToolUse (matcher
"Write") command hooks with plugin-relative paths
3. .claude/commands/review.md into commands/
Update .claude-plugin/plugin.json description to: "Code review toolkit: review skill, format
+ secret hooks, /review command."
Then validate the plugin structure and show me the final tree.
```

Expected output marker: a tree showing `.claude-plugin/plugin.json`, `skills/code-review/SKILL.md`, `hooks/hooks.json`, `commands/review.md`.

Commit it:

```
Initialize git and commit everything with "feat: dashboard-toolkit plugin v0.1.0"
```

Exercise 4: Publish to a Local/Team Marketplace (8 min)

A marketplace is just a git repo (or folder) with a `marketplace.json` index. Build a local one:

```
Create a folder ../team-marketplace containing .claude-plugin/marketplace.json with:
{
"name": "team-marketplace",
"owner": { "name": "Training Team" },
"plugins": [
{ "name": "dashboard-toolkit",
"source": "../dashboard-toolkit-plugin",
"description": "Code review toolkit: review skill, format + secret hooks, /review command."
}
]
}
```

Now install from it - back in your **business-dashboard** project:

```
cd ../business-dashboard
claude
```

```
/plugin marketplace add ../team-marketplace
```

```
/plugin install dashboard-toolkit@team-marketplace
```

Expected output marker: plugin installs; `/plugin` lists dashboard-toolkit as installed, and `/skills` now shows the plugin's code-review skill (marked as coming from the plugin, not the project).

Verify end-to-end:

```
/review server.js
```

Expected output marker: the plugin-delivered command runs the plugin-delivered skill. Anyone on your team can now `marketplace add` + `install` and get your entire toolkit in two commands.

Exercise 5: Tomorrow's Teaser - claude mcp login (2 min)

Plugins can also bundle MCP servers - external tool connections. New since week 28: shell-level auth for MCP servers.

```
claude mcp login --help
```

Expected output marker: usage text for authenticating to MCP servers from the shell (paired with `claude mcp logout`). Don't configure anything yet - Day 3 Lab A starts here.

FINISH MARKER - Success Checklist

- [ ] Inspected a marketplace plugin BEFORE installing (trust model applied)
- [ ] `claude plugin init` scaffold created with valid plugin.json
- [ ] Skill + 2 hooks + command packaged into the plugin
- [ ] Local team marketplace created with marketplace.json
- [ ] Plugin installed back into business-dashboard from the marketplace
- [ ] `/review` works via the plugin
- [ ] `claude mcp login --help` seen
- [ ] Toolkit repo complete: CLAUDE.md + skill + hook + command + plugin manifest

If Stuck

Problem	Recovery
`claude plugin init` not found	Check `claude --version` 2.1.157; run `/doctor` and auto-fix
Marketplace add fails	Path must point at the folder containing `.claude-plugin/marketplace.json`; use an absolute path
Skill collision (project + plugin both define code-review)	Remove the project-local copy from business-dashboard, or rename one - project-level wins
Hooks from plugin not firing	Check `hooks/hooks.json` paths are plugin-relative; `/clear` to reload
Install shows old version	`/plugin uninstall dashboard-toolkit` then reinstall after committing changes

Debrief Questions

1. Why does the plugin trust model say "treat like a developer dependency"?

2. What belongs in a plugin vs in the project's own `.claude/` folder?
3. Which of your team's conventions would you ship as a team plugin first - and who maintains its marketplace?